

PREFACE FOR FACULTY

This lecture establishes the critical connection between linear physical convolution (filtering) and cyclic circular convolution, and formalizes the computational motivation for Fast Fourier Transform (FFT) algorithms.

Pedagogical Strategy:

1. Contrast linear convolution ($y_{lin}[n]$, length $L = L_1 + L_2 - 1$) with circular convolution ($y_{circ}[n]$, length N).
2. Prove that circular convolution equals linear convolution if and only if both sequences are zero-padded to length $N \geq L_1 + L_2 - 1$ (Aliasing-free condition).
3. Derive the exact computational complexity of direct DFT: N^2 complex multiplications and $N(N - 1)$ complex additions.
4. Calculate real floating-point operations (FLOPs): 1 complex multiplication = 4 real multiplications + 2 real additions.
5. Contrast $O(N^2)$ direct computation with $O(N \log_2 N)$ FFT algorithms, demonstrating why FFT is considered one of the top numerical algorithms of the 20th century.

1. LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

1. **Zero-pad** discrete sequences to compute exact linear convolution using circular convolution and the DFT.
2. **Calculate** the exact number of complex and real arithmetic operations required for direct matrix DFT.
3. **Determine** the minimum DFT size N to prevent time-domain wrap-around aliasing.
4. **Quantify** the computational savings factor $\frac{N^2}{\frac{N}{2} \log_2 N} = \frac{2N}{\log_2 N}$ provided by FFT algorithms.

2. MATHEMATICAL FOUNDATIONS

2.1 Linear Convolution via Circular Convolution

Let $x_1[n]$ have length L_1 and $x_2[n]$ have length L_2 . The linear convolution $y_{lin}[n] = x_1[n] * x_2[n]$ has length:

$$L = L_1 + L_2 - 1$$

If we compute an N -point circular convolution $y_{circ}[n] = x_1[n] \circledast_N x_2[n]$:

- **Case 1** ($N < L_1 + L_2 - 1$): Time-domain aliasing occurs due to wrap-around:

$$y_{circ}[n] = \sum_{r=-\infty}^{\infty} y_{lin}[n + rN], \quad 0 \leq n \leq N - 1$$

- **Case 2** ($N \geq L_1 + L_2 - 1$): Zero-padding both sequences to length N completely eliminates wrap-around:

$$y_{circ}[n] = y_{lin}[n], \quad 0 \leq n \leq L_1 + L_2 - 2$$

2.2 Direct DFT Computational Complexity

For an N -point sequence $x[n]$:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk}, \quad k = 0, 1, \dots, N - 1$$

- Each frequency bin $X[k]$ requires N complex multiplications and $N - 1$ complex additions.
- For all N bins:

$$\text{Complex Multiplications } \mu_{\text{direct}} = N \times N = N^2$$

$$\text{Complex Additions } \alpha_{\text{direct}} = N \times (N - 1) = N^2 - N$$

- **Real Arithmetic Operations:** Let $(a + jb)(c + jd) = (ac - bd) + j(ad + bc) \implies 4$ real mults, 2 real adds. Let $(a + jb) + (c + jd) = (a + c) + j(b + d) \implies 2$ real adds.

$$\text{Total Real Multiplications} = 4N^2$$

$$\text{Total Real Additions} = 2N^2 + 2N(N - 1) = 4N^2 - 2N$$

3. WORKED NUMERICAL EXAMPLES

Example 9.1: Linear Convolution via Circular Convolution

Problem: Compute the linear convolution of $x_1[n] = \underset{\uparrow}{1}, 2$ and $x_2[n] = \underset{\uparrow}{2}, 1, 3$ using circular convolution.

Solution:

1. $L_1 = 2, ; L_2 = 3 \implies$ Required Length $N \geq 2 + 3 - 1 = 4$.
2. Zero-pad both sequences to $N = 4$:

$$x_1[n] = \underset{\uparrow}{1}, 2, 0, 0, \quad x_2[n] = \underset{\uparrow}{2}, 1, 3, 0$$

3. Compute $y_c[n] = x_1[n] \otimes_4 x_2[n]$ via circulant matrix:

$$[y[0] \ y[1] \ y[2] \ y[3]] = [2 \ 0 \ 3 \ 1 \ 1 \ 2 \ 0 \ 3 \ 3 \ 1 \ 2 \ 0 \ 0 \ 3 \ 1 \ 2] [1 \ 2 \ 0 \ 0] = [2(1) + 0(2) \ 1(1) + 2(2) \ 3(1) + 1(2) \ 0(1) + 3(2)] = [2 \ 5 \ 5 \ 6]$$

4. Check via polynomial linear convolution: $(1 + 2z^{-1})(2 + z^{-1} + 3z^{-2}) = 2 + z^{-1} + 3z^{-2} + 4z^{-1} + 2z^{-2} + 6z^{-3} = 2 + 5z^{-1} + 5z^{-2} + 6z^{-3}$.
Matches exactly: $y_{lin}[n] = \underset{\uparrow}{2}, 5, 5, 6$.

4. UNIVERSITY EXAMINATION QUESTIONS & MARKING RUBRIC

Question 1 (15 Marks)

(a) Explain how linear convolution can be obtained from circular convolution. What is the minimum sequence length required to avoid aliasing? (5 Marks) **(b)** Compare the computational complexity of evaluating a 1024-point DFT using:

1. Direct matrix computation. (3 Marks)
2. Radix-2 FFT algorithm. (3 Marks) Calculate the percentage reduction in complex multiplications and the execution speedup factor. (4 Marks)

Model Answer & Step-by-Step Marking Rubric:

- **Part (a):**
 - Zero-padding procedure and mathematical proof of wrap-around elimination (3 Marks)
 - Minimum size condition $N \geq L_1 + L_2 - 1$ (2 Marks)
- **Part (b.1):**
 - Direct DFT ($N = 1024$): $\mu_{\text{direct}} = N^2 = 1024^2 = 1,048,576$ complex multiplications
 $\alpha_{\text{direct}} = N(N-1) = 1024 \times 1023 = 1,047,552$ complex additions (3 Marks)
- **Part (b.2):**
 - Radix-2 FFT ($N = 1024 = 2^{10}$): $\mu_{\text{FFT}} = \frac{N}{2} \log_2 N = \frac{1024}{2} \times 10 = 512 \times 10 = 5,120$ complex multiplications
 $\alpha_{\text{FFT}} = N \log_2 N = 1024 \times 10 = 10,240$ complex additions (3 Marks)
- **Part (b.3):**
 - Speedup Factor:

$$\text{Speedup} = \frac{\mu_{\text{direct}}}{\mu_{\text{FFT}}} = \frac{1,048,576}{5,120} = 204.8 \times$$

- Percentage Reduction:

$$\text{Reduction} = \frac{1,048,576 - 5,120}{1,048,576} \times 100$$

(4 Marks)

5. PYTHON VERIFICATION SCRIPT

```
N = 1024
direct_mults = N**2
fft_mults = (N // 2) * int(np.log2(N))
speedup = direct_mults / fft_mults

print(f"Direct DFT Multiplications: {direct_mults:,}")
print(f"Radix-2 FFT Multiplications: {fft_mults:,}")
print(f"Speedup Factor: {speedup:.1f}x")
```